

## Here's a clear mapping of React concepts to Svelte (Svelte 5+ with Runes) equivalents.

Svelte 5's **Runes** (`$state`, `$derived`, etc.) make the comparison much more direct than in Svelte 4.

### Core React Hooks → Svelte Runes

| React Concept                                                           | React Example                                                      | Svelte Equivalent                                     | Svelte Example                                                     | Notes                                                                      |
|-------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>State</b> ( <code>useState</code> )                                  | <pre>const [count, setCount] = useState(0);</pre>                  | <code>\$state</code> rune                             | <pre>let count = \$state(0);</pre>                                 | Direct mutation works ( <code>count++</code> ). No setter function needed. |
| <b>Computed / Memo</b> ( <code>useMemo</code> )                         | <pre>const doubled = useMemo(() =&gt; count * 2, [count]);</pre>   | <code>\$derived</code> rune                           | <pre>let doubled = \$derived(count * 2);</pre>                     | Automatically tracks dependencies. No array needed.                        |
| <b>Side Effects</b> ( <code>useEffect</code> )                          | <pre>useEffect(() =&gt; { ... }, [deps]);</pre>                    | <code>\$effect</code> rune                            | <pre>\$effect(() =&gt; { console.log(count); });</pre>             | Automatic dependency tracking. Runs after DOM update.                      |
| <b>Cleanup in Effect</b>                                                | <pre>useEffect(() =&gt; { return () =&gt; cleanup(); }, []);</pre> | Return cleanup from <code>\$effect</code>             | <pre>\$effect(() =&gt; { return () =&gt; cleanup(); });</pre>      | Very similar.                                                              |
| <b>Layout Effect</b> ( <code>useLayoutEffect</code> )                   | Similar to <code>useEffect</code> but sync                         | <code>\$effect</code> (or custom)                     | Same as above                                                      | Svelte's <code>\$effect</code> is generally post- paint.                   |
| <b>Ref</b> ( <code>useRef</code> )                                      | <pre>const ref = useRef(null);</pre>                               | <pre>let ref = \$state(null);</pre> or plain variable | <pre>let el = \$state(null);</pre>                                 | For DOM refs, use <code>bind:this={el}</code> .                            |
| <b>Callback Memo</b> ( <code>useCallback</code> )                       | <pre>const handler = useCallback(() =&gt; {}, [deps]);</pre>       | Just a function (or <code>\$derived</code> )          | <pre>function handleClick() { ... }</pre>                          | Often unnecessary due to fine-grained reactivity.                          |
| <b>Context</b> ( <code>useContext</code> + <code>createContext</code> ) | Provider + <code>useContext(MyContext)</code>                      | Context API or Stores                                 | <code>setContext</code> / <code>getContext</code> or Svelte stores | Svelte has a built-in context API.                                         |

| React Concept                                                 | React Example            | Svelte Equivalent                            | Svelte Example                             | Notes                                             |
|---------------------------------------------------------------|--------------------------|----------------------------------------------|--------------------------------------------|---------------------------------------------------|
| <b>Reducer</b> ( <code>useReducer</code> )                    | Complex state logic      | <code>\$state</code> + logic or custom store | <code>let state = \$state({ ... });</code> | For complex cases, use a class or writable store. |
| <b>Imperative Handle</b> ( <code>useImperativeHandle</code> ) | Expose methods to parent | <code>\$\$props</code> / events / context    | Expose via context or events               | Less common pattern in Svelte.                    |

## Other Common Concepts

| Category                     | React                                              | Svelte Equivalent                                                                                         | Notes                                                                  |
|------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Props</b>                 | <code>function Component({ name })</code>          | <code>let { name } = \$props();</code>                                                                    | <code>\$props</code> rune in Svelte 5.                                 |
| <b>Children</b>              | <code>{children}</code>                            | <code>{@render children()}</code> or slots                                                                | Svelte uses slots by default.                                          |
| <b>Lifecycle (Mount)</b>     | <code>useEffect(() =&gt; {}, [])</code>            | <code>\$effect(() =&gt; {})</code> or top-level code                                                      | Top-level code runs on init.                                           |
| <b>Lifecycle (Unmount)</b>   | Cleanup in <code>useEffect</code>                  | Return cleanup function from <code>\$effect</code>                                                        | Same as effect cleanup.                                                |
| <b>Styling</b>               | CSS-in-JS, modules, Tailwind                       | Scoped CSS (default), <code>&lt;style&gt;</code> , Tailwind                                               | Svelte's scoped styles are excellent.                                  |
| <b>Events</b>                | <code>onClick={handler}</code>                     | <code>on:click={handler}</code>                                                                           | Native-like.                                                           |
| <b>Conditional Rendering</b> | <code>{condition &amp;&amp; &lt;Comp /&gt;}</code> | <code>{#if condition} ... {/if}</code>                                                                    | More readable.                                                         |
| <b>Lists</b>                 | <code>.map()</code> with <code>key</code>          | <code>{#each items as item (item.id)}</code>                                                              | Key is optional but recommended.                                       |
| <b>Global State</b>          | Redux, Zustand, Context + <code>useReducer</code>  | Svelte Stores ( <code>writable</code> , <code>readable</code> ) or Runes in <code>.svelte.ts</code> files | Stores still work great with <code>\$</code> prefix auto-subscription. |

## Key Philosophical Differences

- **Reactivity Model:**
  - **React:** Virtual DOM + reconciliation + hooks rules + dependency arrays.
  - **Svelte:** Compiler-based, fine-grained reactivity. No VDOM in Svelte 5 (compiles to surgical DOM updates).
- **Mental Model:**
  - React: "Everything re-renders unless I memoize."
  - Svelte: "Only the things that actually change update."
- **Boilerplate:**

- Svelte generally requires **much less** boilerplate once you're used to runes.

## Quick Code Comparison Example (Counter)

### React:

```
function Counter() {
  const [count, setCount] = useState(0);
  const doubled = useMemo(() => count * 2, [count]);

  useEffect(() => {
    console.log('Count changed:', count);
    return () => console.log('Cleanup');
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>{count} (doubled: {doubled})
</button>;
}
```

### Svelte 5:

```
<script>
  let count = $state(0);
  let doubled = $derived(count * 2);

  $effect(() => {
    console.log('Count changed:', count);
    return () => console.log('Cleanup');
  });
</script>

<button on:click={() => count++}>
  {count} (doubled: {doubled})
</button>
```

Would you like me to expand this into a full side-by-side component examples (forms, todo list, data fetching, etc.)?